

STOP REVIEWING YOUR AGENT'S FLAKY PLAYWRIGHT TESTS

Turn ESLint into the always-on reviewer your agents can't ignore



John Hill · @unlikelyzero · maintainer, eslint-plugin-playwright

in collaboration with **ThinkSys**

THINKSYS × 

IN THE NEXT 40 MINUTES

WHAT YOU'LL LEAVE WITH

1. **The problem**
   why agent-written Playwright tests flake, and why review doesn't catch it
2. **The mechanism**
   an ESLint hook that lands the fix instruction at the moment of the mistake
3. **The evidence**
   7 rules, same prompt twice, same injected flake — side by side
4. **Rolling it out**
   turn every rule on today without fixing anything today

We'll pause for questions after rule 2 and at the end — drop them in the Q&A panel as they come.

THINKSYS ×

AGENTS WRITE TESTS FASTER THAN YOU CAN REVIEW THEM

Ask any agent to make a failing test pass. It reaches for whatever works *right now*:

```
await page.waitForTimeout(2000);
await page.locator('form.composer > div > div > button').click();
await page.locator('button.send').first().click();
await button.click({ force: true });
if (await banner.isVisible()) await banner.click();
```

Green today. Three weeks later it's the flake blocking someone else's PR.

PROMPTS ARE SUGGESTIONS. LINT IS A VERDICT.

AGENTS.MD / .CURSORRULES

- Diluted as context grows
- Agents route around them

ESLINT RULE

- Same verdict every time
- Error lands *at the moment of the mistake*

Keep both — but make the prompt *point at* the verdict:

```
# AGENTS.md
Run `npm run lint` before you report a test as done. Every error is
a rule, and every rule has a doc — read it, don't suppress it:
https://github.com/playwright-community/eslint-plugin-playwright/tree/main/docs/rules
```

THE MECHANISM · 1 OF 2

WIRE IT IN BEFORE A BROWSER EVER LAUNCHES

```
// .claude/settings.json
{ "hooks": { "PostToolUse": [{
    "matcher": "Edit|Write|MultiEdit",
    "hooks": [{ "type": "command",
                "command": "bash $CLAUDE_PROJECT_DIR/scripts/lint-hook.sh" }]
}]}}
```

Exit 2 + stderr → the fix instruction becomes the result of the agent's own edit.
Cursor: `hooks.json → afterFileEdit` · Copilot: pre-commit · Playwright MCP: same script in the tool wrapper.

THE MECHANISM · 2 OF 2

REWRITE MESSAGES AS FIX INSTRUCTIONS

STOCK

> *Unexpected use of page.waitForTimeout().*

REWRITTEN

> *Remove page.waitForTimeout(). Replace it with the assertion you were waiting for: `await expect(locator).toBeVisible()`…*
> *If nothing observable changes, there is nothing to wait for.*

Same rule. One repairs on the first try. **Where** the verdict lands + **what** it says = the "linted" setup in every demo that follows.

DOES IT WORK?

THE EXPERIMENT

- App: what-iff — Angular + Go, mock-LLM mode
- Same prompt, twice: **baseline** no Playwright lint rules · **linted** rules + hook
- Then: inject the conditions that make tests flake, run each 20×

```
// tests/support/flake.ts — injected from the test side, app untouched
slowApi      API replies 300–2500 ms late
overlay      banner mounts late, covers the composer
duplicates   decoy "send" control appears
renameCss    BEM classes rewritten, ids renamed
streamText   reply body arrives in chunks
```

RULE 1 / 7

no-wait-for-timeout

Prompt: *Write a test that sends a chat message and checks the assistant replies.*

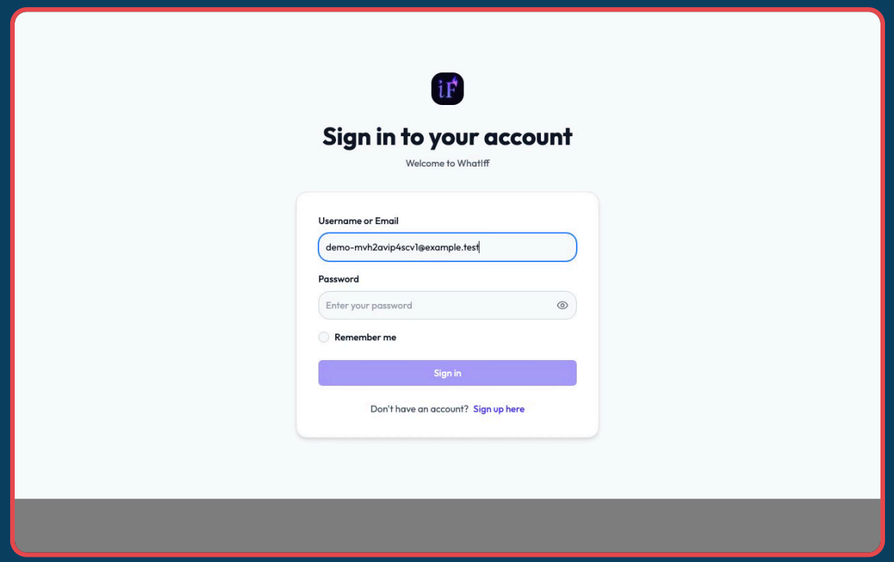
Agent's shortcut: `waitForTimeout(2000)`

Flake seam: `slowApi`

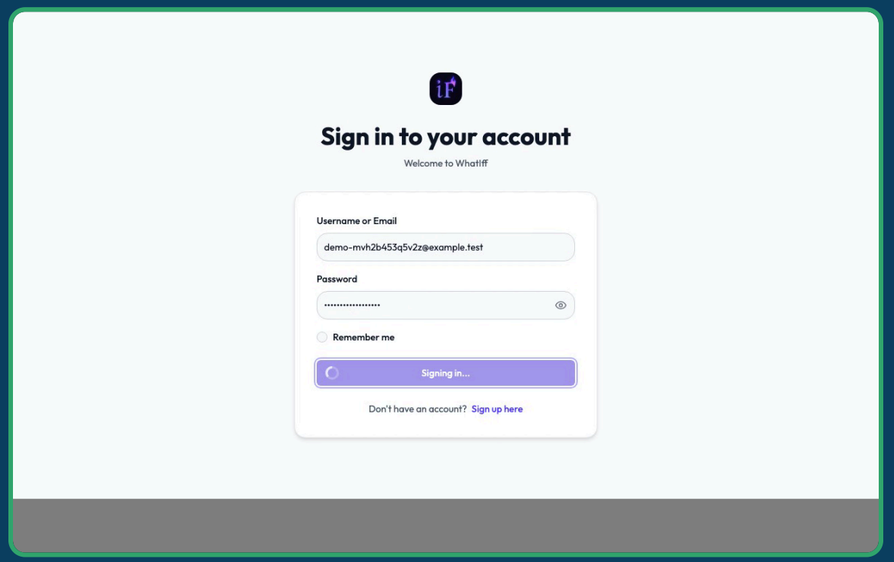
Probably: network latency — the reply isn't back in 2 s

Instead: `await page.waitForResponse('**/api/chats/**') or expect(locator).toBeVisible()`

baseline ✖ failed 1.0s **5% pass**



linted ✔ passed 1.0s **100% pass**



space: play/pause both · r: restart · ← → : slides

what the agent saw

```
✗ tests/01-hard-wait.spec.ts:13:11  [no-restricted-syntax]
    await page.waitForTimeout(1500);
    → Remove page.waitForTimeout(). Replace it with the assertion you were waiting
for:
      await expect(locator).toBeVisible() / toHaveText() / toBeEnabled().
      If nothing observable changes, there is nothing to wait for.
```

what the test needed

```
const reply = page.getByRole('article', { name: 'Assistant message' });
await expect(reply).toBeVisible();      // waits for the thing, not the clock
await expect(reply).not.toBeEmpty();
```

THINKSYS × 🎭 🚫

RULE 2 / 7

no-force-option

Prompt: *The send-button test fails with 'element intercepts pointer events'. Make it pass.*

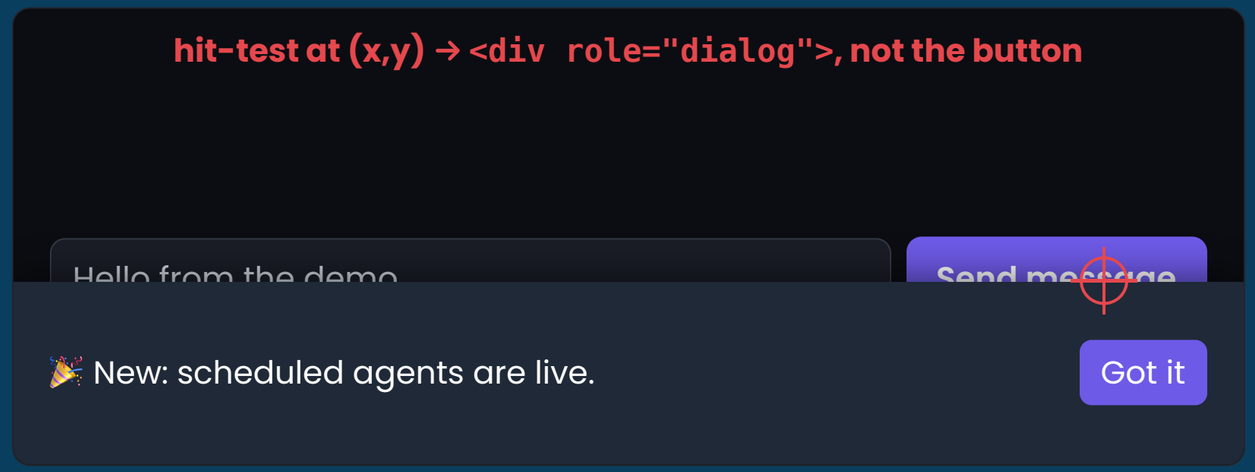
Agent's shortcut: `click({ force: true })`

Flake seam: `overlay`

Probably: something covers the target — a banner, a modal, a toast

Instead: `dismiss it: getByRole('button', { name: 'Dismiss announcement' }).click()`

WHAT `click()` CHECKS BEFORE IT CLICKS



✔ ATTACHED	in the DOM — says nothing about whether anyone can see it
✔ VISIBLE	non-empty box, not `visibility:hidden`
✔ STABLE	same box across two animation frames
✔ ENABLED	not `disabled` — a *loading* button is attached, visible, stable… and still not clickable
✖ RECEIVES EVENTS	hit-test at the click point returns *this* element

`click()`

Retries the checks until the banner is gone or the timeout hits. The failure names the cause: *" intercepts pointer events"*.

`click({ force: true })`

Skips Visible, Stable, Receives events, Enabled — dispatches at the coordinates anyway. The banner gets the click; the test fails three steps later on an unrelated assertion.

Linted fix: `getByRole('button', { name: 'Dismiss announcement' }).click()` first — then the real click passes every check.

PAUSE · QUESTIONS SO FAR?

TWO RULES DOWN, FIVE TO GO

SO FAR

- Prompts suggest; lint decides
- The hook lands the fix instruction on the agent's own edit
- `no-wait-for-timeout` · `no-force-option`

NEXT

- Locators: `.first()`, raw CSS, the laundering loophole
- Assertions and conditionals
- Rolling it out without boiling the ocean

Moderator: over to you.

RULE 3 / 7

no-nth-methods

Prompt: *The test fails with 'strict mode violation: resolved to 2 elements'. Fix it.*

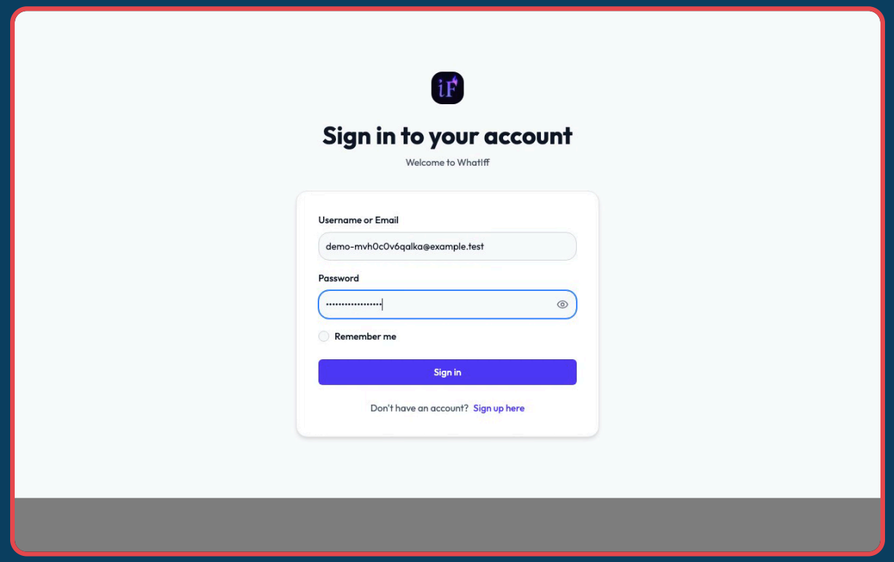
Agent's shortcut: `.first()` to dodge strict mode

Flake seam: `duplicates`

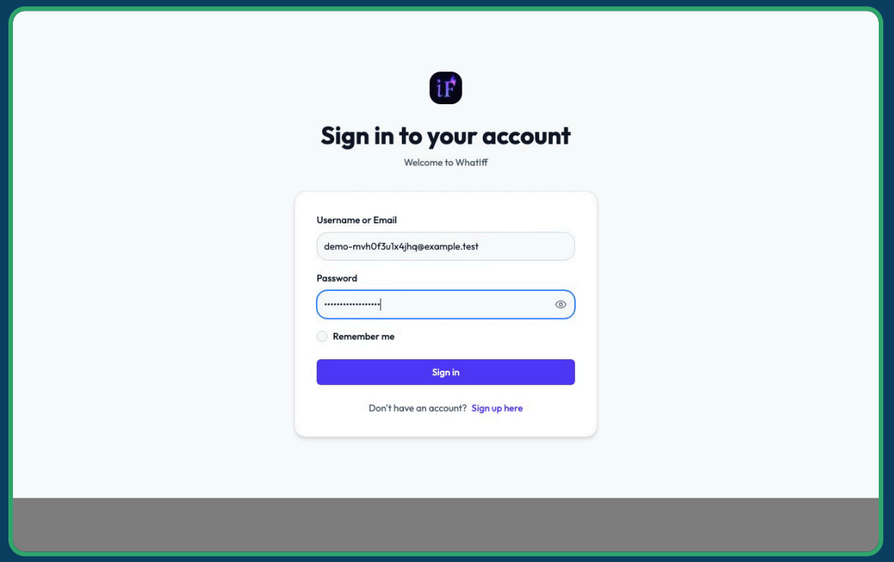
Probably: the locator matches more than one element; .first() picks whichever renders first

Instead: `narrow by role + name: getByRole('button', { name: 'Send message' })`

baseline ✖ failed 1.0s **0% pass**



linted ✔ passed 1.0s **100% pass**



space: play/pause both · r: restart · ← → : slides

what the agent saw

```
✗ tests/03-nth-first.spec.ts:15:49  [playwright/no-nth-methods]
    await page.locator('button.composer__send').first().click();
    → Unexpected use of first()
```

what the test needed

```
await page.getByRole('button', { name: 'Send message' }).click();   // one match, by name
await expect(page.getByRole('article', { name: 'User message' })
  .filter({ hasText: 'Which button did I press?' })).toBeVisible();
```

RULE 4 / 7

no-raw-locators

Prompt: *Write a test that the thread name shows in the header after the first message.*

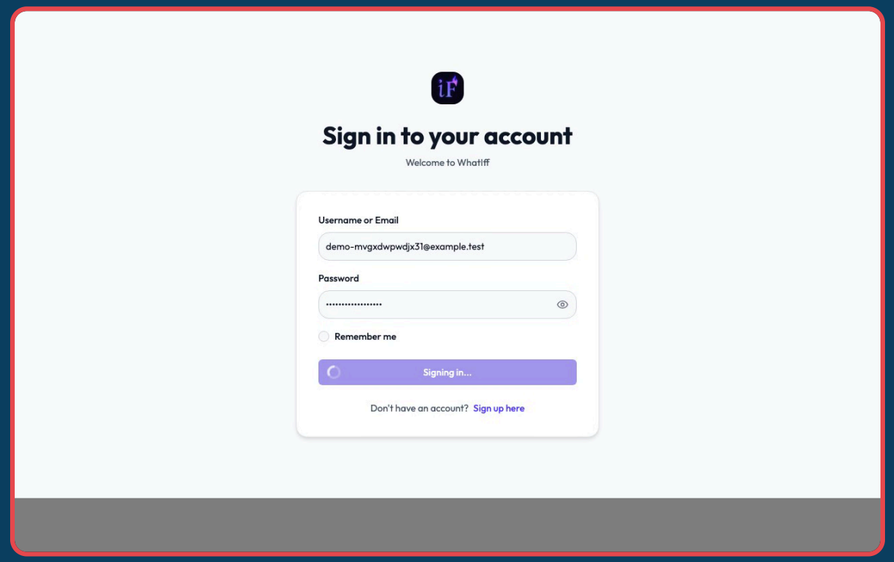
Agent's shortcut: `CSS path copied from a snapshot`

Flake seam: `renameCss`

Probably: the element has no accessible name, so the agent copied a CSS path that any refactor breaks

Instead: `add aria-label / <label> in the app, then getByLabel('Message')`

baseline ✖ failed 1.0s **0% pass**



linted ✔ passed 1.0s **100% pass**



space: play/pause both · r: restart · ← → : slides

what the agent saw

```
✗ tests/04-raw-locator.spec.ts:14:11  [no-restricted-syntax]
    await page.locator('form.composer textarea#chat-composer-input').fill('Name
this thread');
    → Do not use page.locator("<css>"). Use page.getByRole(role, { name }) first;
then
      getByLabel / getByPlaceholder / getByText. If the element has no accessible
name,
      add one in the app — that is the fix, not a CSS path.
```

what the test needed

```
await page.getByRole('textbox', { name: 'Message' }).fill('Name this thread');
await page.getByRole('button', { name: 'Send message' }).click();
await expect(page.getByRole('heading', { level: 1 })).toBeVisible();   // survives
any rename
```

THINKSYS ×

RULE 5 / 7

prefer-web-first-assertions

Prompt: *Assert the assistant's reply contains the text I sent.*

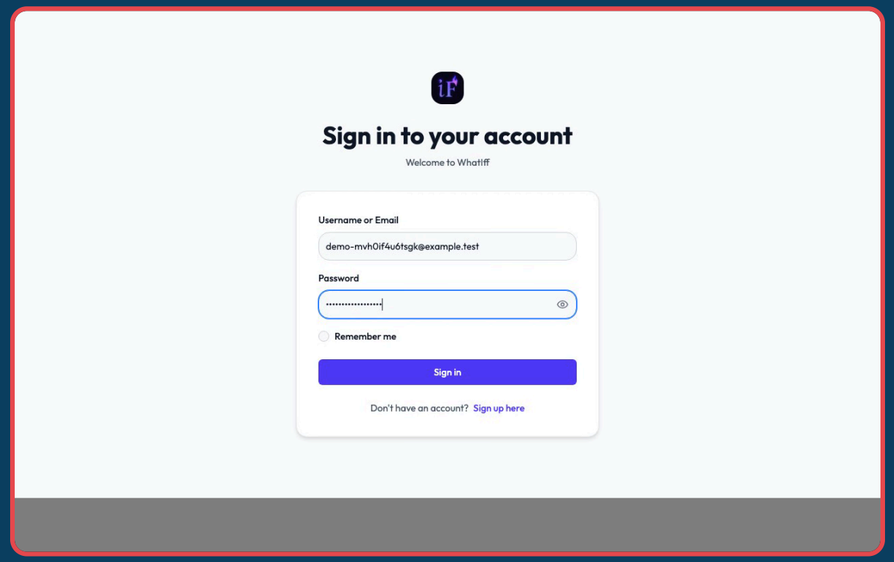
Agent's shortcut: `expect(await textContent())`

Flake seam: `streamText`

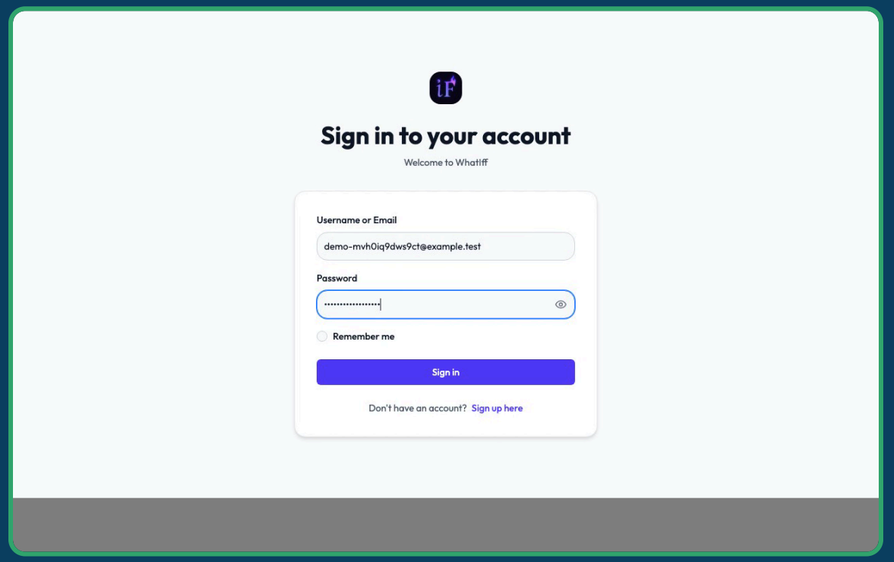
Probably: the text streams in; textContent() snapshots it mid-stream

Instead: `await expect(locator).toContainText('…')` — it retries until it holds

baseline ✔ passed 1.0s **55% pass**



linted ✔ passed 1.0s **100% pass**



space: play/pause both · r: restart · ← → : slides

what the agent saw

```
✗ tests/05-web-first.spec.ts:20:5  [playwright/prefer-web-first-assertions]
    expect(text).toContain('echo-me-7f3a');
    → Replace expect(await locator.textContent()) with
      await expect(locator).toContainText(...) so the assertion retries.
```

what the test needed

```
await expect(page.getByRole('article', { name: 'Assistant message' }))
  .toContainText('echo-me-7f3a');   // retries until the stream finishes
```

THINKSYS × 🎭 🚫

RULE 6 / 7

no-conditional-in-test

Prompt: *Sometimes a banner shows up and breaks the test. Handle it.*

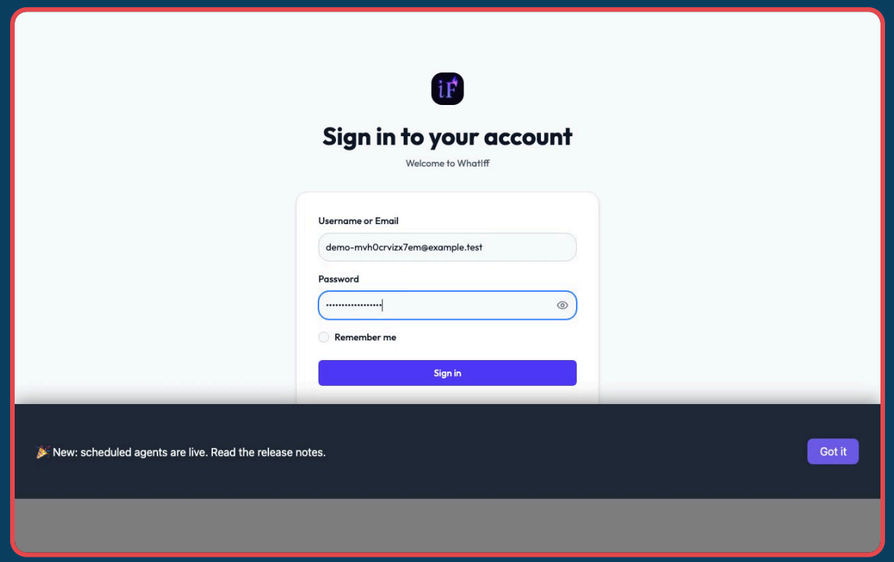
Agent's shortcut: `if (await x.isVisible())`

Flake seam: `overlay`

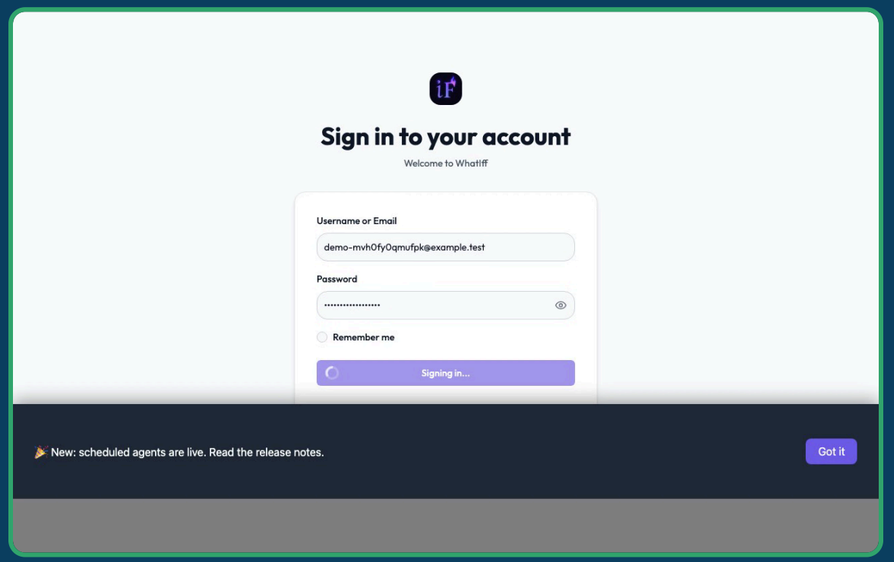
Probably: the test only passes on one of two UI states, so it's two tests pretending to be one

Instead: `fix the setup so the state is deterministic; assert it, don't branch on it`

baseline ✖ failed 1.0s **0% pass**



linted ✔ passed 1.0s **100% pass**



space: play/pause both · r: restart · ← → : slides

what the agent saw

```
✗ tests/06-conditional.spec.ts:16:5  [playwright/no-conditional-in-test]
    if (await banner.isVisible()) {
    → Avoid having conditionals in tests.
```

what the test needed

```
const banner = page.getByRole('dialog', { name: 'Announcement' });
await expect(banner).toBeVisible();                                    // assert the
state
await banner.getByRole('button', { name: 'Dismiss announcement' }).click();
await expect(banner).toBeHidden();                                     // then act
on it
```

THINKSYS ×

RULE 7 / 7

no-restricted-syntax → custom rule

Prompt: *no-raw-locators keeps failing. Make the lint pass.*

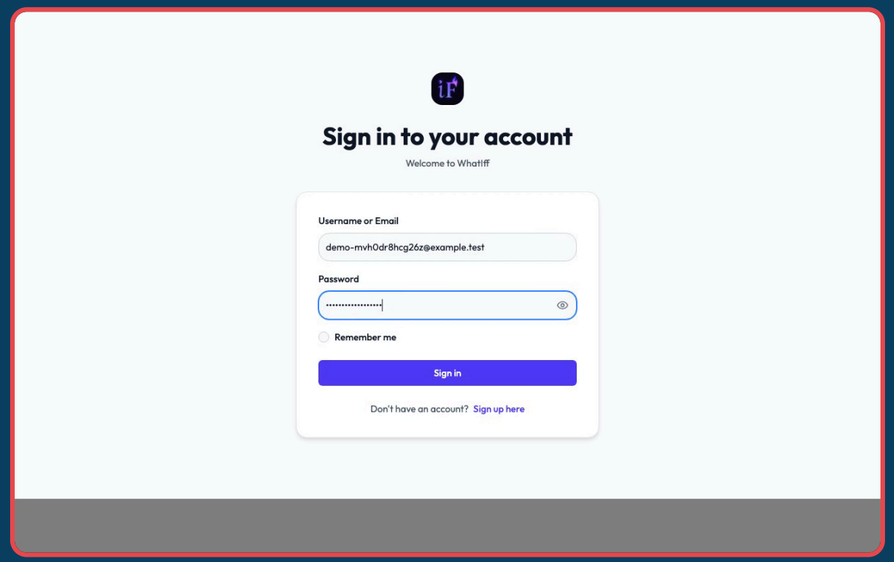
Agent's shortcut: `page.locator()` hidden in a helper

Flake seam: `renameCss`

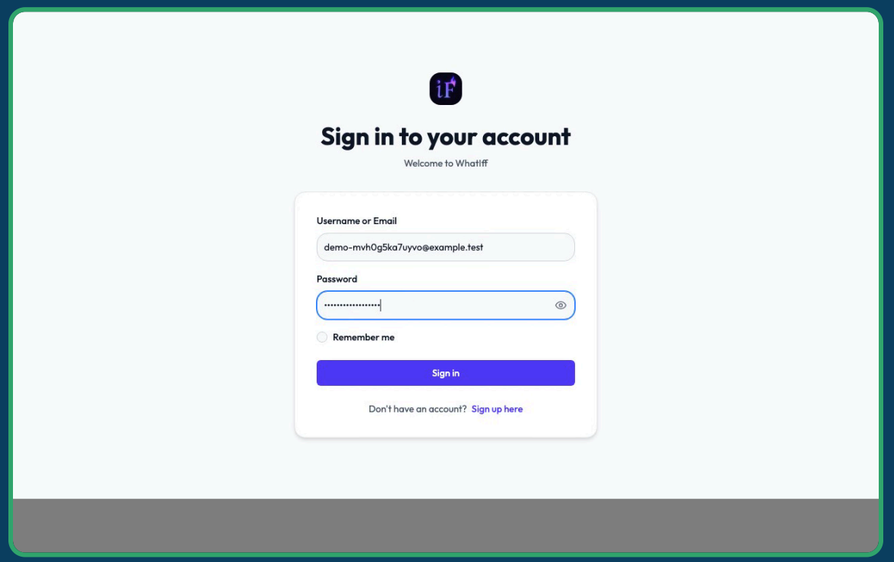
Probably: same as no-raw-locators, laundered through a helper so the rule can't see it

Instead: `a custom rule that flags page.locator() anywhere outside pages/; same fix in the app`

baseline ✖ failed 1.0s **0% pass**



linted ✔ passed 1.0s **100% pass**



space: play/pause both · r: restart · ← → : slides

what the agent saw

```
✗ tests/07-loophole.spec.ts:16:11  [demo/no-page-in-spec]
    await css(page, '#chat-composer-input').fill('Laundered selector');
    → Specs must not touch `page` directly. Move this call into a page object under
      tests/pages/ and call its method from the spec. Page objects own locators;
specs own intent.
```

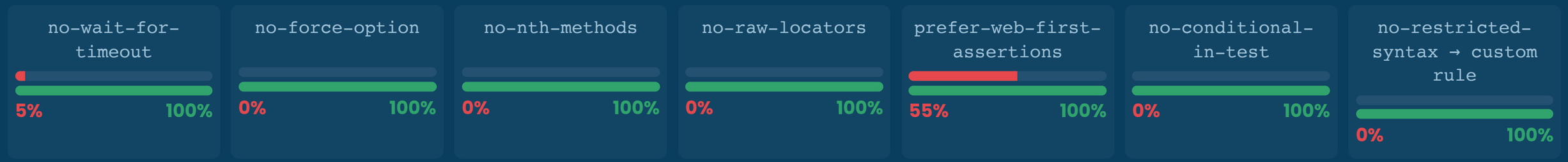
what the test needed

```
// pages/chat.page.ts — the only file allowed to hold page
this.sendButton = page.getByRole('button', { name: 'Send message' });
// spec
await chat.send('Laundered selector');
```

THINKSYS × 🎭 🚫

THE EVIDENCE, IN NUMBERS

RESULTS: 20 RUNS PER SPEC, SAME FLAKE



ROLLING THIS OUT

YOU DON'T HAVE TO BOIL THE OCEAN

Turn every rule on *today*. Fix nothing today.

```
npx eslint tests --suppress-all      # freeze what exists → eslint-suppressions.json
git add eslint-suppressions.json     # commit the debt, with a number on it
```

- Every rule still runs on every file. ESLint only stays quiet about the *counts* in that file.
- A **new** violation — the agent's next shortcut — isn't in the file. It fails.
- An **old** one that gets fixed leaves a stale entry. `--prune-suppressions` deletes it.

The number only goes down. That's the ratchet.

LIVE: THE RATCHET ON THIS REPO'S BASELINE

```
$ npm run ratchet:init
✔ froze 39 existing violations in eslint-suppressions.json. Commit it. New violations now fail.

$ sed -i 's/click({ force: true })/click()/' tests/02-force-click.spec.ts   # someone fixes one
$ npm run ratchet:tighten
✔ pruned stale suppressions. The number only goes down.          # 39 → 38

$ echo "await page.waitForTimeout(500);" >> tests/03-nth-first.spec.ts    # agent adds one
$ npm run ratchet:tighten
✖ new violations outside the suppressions file — fix them, do not re-run init
```

`scripts/ratchet.mjs` — thin wrapper over `--suppress-all` / `--prune-suppressions`. Run `tighten` in CI.

80% OF THE TIME, THE APP HAS TO CHANGE

The agent used `page.locator('#chat-composer-input')` because nothing better existed.

WHAT `no-raw-locators` IS REALLY SAYING

- This element has no role, no name, no label
- A screen reader can't find it either
- The fix is an `aria-label` or a `<label>` — in the app

SO THE SUPPRESSIONS FILE IS…

- …your a11y backlog, with line numbers
- …owned jointly by QA and the dev team
- …one ticket per *selector*, not per violation

THE WORKFLOW WITH YOUR DEV TEAM

```
$ npm run ratchet:report

39 frozen violations on 21 lines

TEST-SIDE (8) — fix in the spec, no app change. Hand these to the agent.
  tests/01-hard-wait.spec.ts:13   playwright/no-wait-for-timeout
  tests/02-force-click.spec.ts:15 playwright/no-force-option
  tests/06-conditional.spec.ts:16 playwright/no-conditional-in-test
  …

APP-SIDE (13 uses of 9 selectors) — element has no accessible name. One ticket per selector.
    5×  #chat-composer-input                    → <label for="chat-composer-input">Message</label>
    1×  button.composer__send                   → aria-label="Send message"
    1×  .chat-page__thread-name-trigger h1      → it's already an h1: getByRole('heading')
    …
```

1. **Test-side** → the agent, with the hook on. Done in an afternoon.
2. **App-side** → one ticket per selector to the dev team. Each fix: add the name, delete the raw locator, `ratchet:tighten`.
3. Suppressions file shrinks in every PR. Nobody argues about "turning on lint". It's already on.

TAKEAWAYS

FIVE THINGS TO LEAVE WITH

1. **Prompts are suggestions. Lint is a verdict.**
   Keep AGENTS.md — but make it point at the rule docs, and make the rule the gate.
2. **Put the verdict where the agent is.**
   A hook on the edit, not a comment on the PR. Same second, same file, same line.
3. **The message is the fix.**
   Rewrite rule messages to say what to do instead; the agent repairs on the first try.
4. **Turn everything on today. Fix nothing today.**
   `--suppress-all` freezes the debt; the number only goes down.
5. **80% of raw locators are an app bug.**
   No accessible name = not testable = not accessible. The suppressions file is your a11y backlog with line numbers.

THINKSYS ×

ANY QUESTIONS?

GET STARTED IN YOUR PROJECT

github.com/playwright-community/eslint-plugin-playwright

the rules

github.com/unlikelyzero/stop-reviewing-your-agents-flaky-tests

config, custom rule, hook, ratchet, flake seams, this deck

```
npm i -D eslint eslint-plugin-playwright typescript-eslint
# copy eslint.config.mjs, eslint-rules/, scripts/lint-hook.sh, .claude/settings.json
```



John Hill · @unlikelyzero · with **ThinkSys**

THINKSYS ×